

PONTIFICIA UNIVERSIDAD CATÓLICA DE
VALPARAÍSO

FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA INFORMÁTICA

PokeHash

Gestor Táctico de Pokémon

Informe Final de Proyecto

Asignatura: Estructuras de Datos

Integrantes:

Cristóbal Sazo
Sebastián Riveros
Simón Guzmán
Bruno Orellana

Profesor:

Ignacio Araya

Fecha:

21 de junio de 2026

Valparaíso, Chile

Índice

1. Introducción	1
1.1. Contextualización y Propósito	1
1.2. Visión General	1
1.3. Generación de Interés e Innovación	2
2. Descripción de la Aplicación e Interacción	2
2.1. Mapa de Navegación	2
2.2. Flujo de Interacción Detallado	3
2.2.1. Buscar por Nombre	3
2.2.2. Buscar por Número de Pokédex	3
2.2.3. Buscar con Filtro por Generación y Tipo	4
2.2.4. Agregar Pokémon al Equipo y Deshacer	4
3. Arquitectura de Datos y Estructuras	5
3.1. Modelamiento de Entidades	5
3.1.1. Entidad <code>Pokemon</code>	5
3.1.2. Entidad <code>Equipo</code>	5
3.1.3. Enumeración <code>PokemonType</code>	6
3.2. Instancias y Selección de TDAs	6
3.3. Estructuras de Datos Físicas	7
3.3.1. Mapas: Tabla Hash con Encadenamiento Separado	7
3.3.2. Pila del Equipo: Arreglo Estático de Punteros	8
3.3.3. Grafo de Tipos: Matriz de Adyacencia Estática	9
4. Diseño de la Solución Algorítmica	9
4.1. Lógica Operacional por Funcionalidad	9
4.1.1. Cargar Catálogo (<code>cargar_data</code>)	9
4.1.2. Cargar Matriz de Efectividades (<code>cargar_matriz_debilidades</code>)	10
4.1.3. Buscar por Nombre (<code>buscar_por_nombre</code>)	10
4.1.4. Buscar por Número de Pokédex (<code>buscar_por_id</code>)	10
4.1.5. Buscar con Filtro por Generación y Tipo (<code>buscar_por_filtro_gen_tipo</code>)	11
4.2. Algoritmo Core: Ranking Top 10 (<code>mejores10PorStat</code>)	11
5. Análisis de Complejidad	12
5.1. Tabla de Complejidad Temporal y Espacial	12
5.2. Justificación de Bajo Nivel	13

5.2.1.	<code>buscar_por_nombre()</code> y <code>buscar_por_id()</code> : $O(1)$ promedio . . .	13
5.2.2.	<code>buscar_por_filtro_gen_tipo()</code> : $O(N)$ inherente e irreducible .	13
5.2.3.	<code>mejores10PorStat()</code> : $O(N)$ con ventana de inserción constante	13
5.2.4.	<code>deshacer()</code> : $O(1)$ estricto	14
5.2.5.	<code>liberar_pokedex()</code> : $O(N)$ con doble responsabilidad de propiedad	14
6.	Aspecto Desafiante	14
6.1.	Gestión Manual de Memoria Dinámica con Doble Modelo de Propiedad	14
6.2.	Diseño de la Función Hash con Insensibilidad a Mayúsculas	15
6.3.	Coordinación Precisa de la Indexación Dual	16
7.	Planificación y Contribución del Equipo	16
7.1.	Distribución de Roles y Responsabilidades	16
7.2.	Lista de Tareas Realizadas por Integrante	18
7.3.	Carta Gantt	19
8.	Conclusión	19

1. Introducción

1.1. Contextualización y Propósito

El universo Pokémon se caracteriza por una compleja red de interacciones estratégicas entre 18 tipos elementales, estadísticas de combate y sinergias grupales. Con el auge del formato competitivo formal — VGC oficial de *The Pokémon Company* y la plataforma comunitaria *Smogon* — la planificación táctica se ha convertido en un requisito indispensable. Al construir un equipo de seis integrantes, el usuario debe equilibrar seis estadísticas base (HP, Ataque, Defensa, Ataque Especial, Defensa Especial, Velocidad) y simultáneamente mitigar vulnerabilidades elementales frente a los 18 tipos existentes.

Actualmente, este proceso adolece de un vacío práctico: los jugadores se ven obligados a recurrir a wikis web, hojas de cálculo o calculadoras externas. Estas soluciones fragmentan la experiencia, dependen de conexión a Internet y no ofrecen una interfaz unificada para el análisis táctico. En respuesta a esta necesidad surge **PokeHash**, una aplicación de consola *offline* que unifica en una sola herramienta un catálogo completo de Pokémon con motores de búsqueda de tiempo constante y un gestor táctico de equipo.

1.2. Visión General

PokeHash procesa y organiza una base de datos de más de 1.100 Pokémon, incluyendo formas regionales y variedades. Sus principales funcionalidades son:

- **Carga Masiva de Datos:** Inicialización desde `Pokemon.csv` (~ 1.100 registros) y la matriz de efectividades `Tabla.csv` (18×18 entradas).
- **Búsqueda Instantánea por Nombre:** Consulta de ficha técnica en tiempo $O(1)$ promedio mediante Tabla Hash, con insensibilidad a mayúsculas.
- **Búsqueda Instantánea por ID:** Consulta por número de Pokédex en tiempo $O(1)$ promedio mediante un segundo Mapa Hash dedicado.
- **Filtrado por Generación y Tipo:** Listado de Pokémon que cumplan simultáneamente dos criterios: generación de origen (1–9) y tipo elemental.
- **Ranking Top 10:** Clasificación de los mejores 10 Pokémon por una o dos estadísticas base, con filtros opcionales de generación o tipo.
- **Gestión de Equipo con Deshacer:** Pila de hasta 6 integrantes con operación Undo en $O(1)$ exacto.

Limitaciones: La aplicación no simula batallas en tiempo real, no gestiona múltiples equipos en memoria de forma persistente, y no incluye interfaz gráfica nativa en su núcleo C.

1.3. Generación de Interés e Innovación

La innovación principal de PokeHash es su arquitectura de **indexación dual**: dos Tablas Hash personalizadas (`pokedex` por nombre y `pokedex_by_id` por ID) que garantizan acceso en tiempo $O(1)$ promedio para ambos tipos de búsqueda. El sistema de tipos elementales se modela como un **Grafo Dirigido Ponderado** implementado como Matriz de Adyacencia estática de 18×18 , con consultas de efectividad en $O(1)$. La combinación de estos TDAs con una Pila estática para la gestión táctica convierte a PokeHash en una solución de alto rendimiento computacional construida íntegramente en C, sin librerías externas.

2. Descripción de la Aplicación e Interacción

2.1. Mapa de Navegación

La aplicación se organiza en un menú principal con dos submenús funcionales:

- **Menú Principal**
 - **1. Explorar Pokédex**
 - 1. Buscar por Nombre
 - 2. Buscar por Número de Pokédex
 - 3. Buscar con Filtro (Generación y Tipo)
 - 4. Mejores 10 por Estadística
 - 5. Volver al Menú Principal
 - **2. Gestión Estratégica de Equipo**
 - 1. Agregar Pokémon al Equipo
 - 2. Ver Equipo Actual
 - 3. Deshacer Última Adición
 - 4. Volver al Menú Principal
 - **3. Exportar Equipo a Archivo**
 - **4. Salir**

2.2. Flujo de Interacción Detallado

2.2.1. Buscar por Nombre

Escenario ideal:

1. El usuario selecciona la opción 1 (Explorar Pokédex) y luego 1 (Buscar por Nombre).
2. El sistema imprime: `Ingrese el nombre del Pokémon a buscar:`.
3. El usuario ingresa `Charizard` (la búsqueda ignora mayúsculas, por lo que `charizard` también funciona).
4. El sistema ejecuta la búsqueda en la Tabla Hash y muestra la ficha técnica completa: nombre, ID, Tipo 1, Tipo 2, HP, Ataque, Defensa, Ataque Especial, Defensa Especial, Velocidad, Total de Estadísticas y Generación.
5. El sistema pausa con `"Presione ENTER para continuar..."` y devuelve el control al submenú.

Escenarios de error:

- Pokémon no encontrado: `"[!] Pokémon 'X' no encontrado en la Pokédex."`.
- Entrada en blanco o solo espacios: `"[Error] Nombre inválido."`.

2.2.2. Buscar por Número de Pokédex

Escenario ideal:

1. El usuario ingresa 25.
2. El sistema convierte el entero a cadena (`"25"`) y ejecuta la búsqueda en el mapa secundario `pokedex_by_id`, mostrando la ficha de Pikachu.

Escenarios de error:

- ID inexistente: `"[!] Pokémon con número X no encontrado en la Pokédex."`.
- Carácter no numérico: `"[Error] Entrada no válida."` y limpieza del buffer de entrada con un bucle `while((c = getchar()) != '\n')`.

2.2.3. Buscar con Filtro por Generación y Tipo

Escenario ideal:

1. El sistema solicita la generación. El usuario ingresa 1.
2. El sistema solicita el tipo. El usuario ingresa **Fire**.
3. El sistema itera el mapa completo y muestra una tabla con ID, Nombre, Tipo 1, Tipo 2 y BST de todos los Pokémon de Generación 1 que posean el tipo **Fire** en cualquiera de sus dos slots de tipo.
4. Al final se imprime el total de coincidencias.

Escenarios de error:

- Sin coincidencias: se imprime "Total: 0 Pokémon encontrados."
- Tipo en blanco: "[Error] Tipo inválido."

2.2.4. Agregar Pokémon al Equipo y Deshacer

Escenario ideal:

1. El usuario selecciona opción 2 → 1 (Agregar). Ingresa **Charizard**".
2. El sistema busca en la Tabla Hash. Tras encontrarlo, confirma: "[!] ¡Charizard ha sido añadido exitosamente! (Espacios ocupados: 1/6)".
3. El usuario decide revertir y selecciona opción 2 → 3 (Deshacer).
4. El sistema responde: "[!] Última acción revertida. El equipo vuelve a tener 0 integrante(s)".

Escenarios de error:

- Equipo lleno (tope = 6): "[Error] El equipo está completo (máximo 6 Pokémon)."
- Deshacer con equipo vacío: "[!] El equipo ya está vacío. No hay nada que deshacer."
- Pokémon no encontrado: "[!] Pokémon 'X' no encontrado en la Pokédex."

2.2.5. Analizar Debilidades del Equipo

Escenario ideal:

1. El usuario tiene un equipo con al menos un Pokémon y selecciona la opción **Analizar Debilidades**.
2. Por cada integrante, el sistema muestra sus debilidades elementales:
 - Débil al tipo Ice (x2)" para multiplicador $\times 2$.
 - "[CRÍTICO] ¡Débil x4 al tipo Rock!" para multiplicador $\times 4$.
3. Tras el análisis individual, el sistema emite alertas globales si 2 o más integrantes comparten debilidad al mismo tipo: "[ALERTA] X integrantes de tu equipo son débiles al tipo: Y".
4. Si no hay alertas, se confirma: "[OK] ¡Tu equipo tiene una excelente sinergia de tipos!".

Escenarios de error:

- Equipo vacío: "[!] El equipo está vacío. No hay Pokémon para analizar."
- Tipo no reconocido en la enumeración: `obtener_indice_tipo` retorna -1 y ese tipo se omite sin abortar el análisis.

3. Arquitectura de Datos y Estructuras

3.1. Modelamiento de Entidades

3.1.1. Entidad Pokemon

La entidad fundamental del sistema modela a cada criatura del catálogo. Está definida en `pokehash.h`:

```

1 typedef struct {
2     int id;           // Numero de Pokedex (identificador numerico
                       // unico por especie)
3     char nombre[100]; // Nombre unico (incluye forma regional si
                       // aplica)
4     char tipo1[20];   // Tipo elemental primario (siempre presente)
5     char tipo2[20];   // Tipo elemental secundario (cadena vacia si
                       // es monotipo)

```



```

6   int total_stats; // BST: suma de las seis estadísticas base
7   int hp;          // Puntos de vida base
8   int ataque;      // Ataque físico base
9   int defensa;     // Defensa física base
10  int ataque_esp;   // Ataque especial base
11  int defensa_esp;  // Defensa especial base
12  int velocidad;   // Velocidad base
13  int gen;         // Generación de origen (1 a 9)
14 } Pokemon;

```

Listing 1: Definición del `struct Pokemon` en `pokehash.h`

El campo `nombre` puede contener el nombre base ("Pikachu") o un nombre compuesto que integra la forma regional. La lógica de generación de nombres únicos en `cargar_data()` aplica tres reglas: (1) si el campo *Form* está vacío, se usa el nombre base; (2) si *Form* ya incluye el nombre base (ej: "Mega Venusaur"), se usa directamente; (3) si no, se construye el formato "Nombre (Forma)" (ej: "Meowstic (Male)"). Esto garantiza la unicidad de clave en la Tabla Hash.

3.1.2. Entidad Equipo

La entidad gestora del equipo almacena hasta 6 punteros a estructuras `Pokemon`:

```

1 typedef struct {
2     Pokemon* integrantes[6]; // Arreglo estático de 6 punteros a
                               // Pokemon
3     int tope;                // Índice del próximo slot libre (0 =
                               // vacío, 6 = lleno)
4 } Equipo;

```

Listing 2: Definición del `struct Equipo` en `pokehash.h`

El campo `tope` actúa como el índice de cima de una Pila: al agregar, se asigna `integrantes[tope] = p` y se incrementa `tope`; al deshacer, solo se decrementa `tope`. Cuando `tope == 0` el equipo está vacío; cuando `tope == 6` está completo.

3.1.3. Enumeración `PokemonType`

Para indexar la Matriz de Adyacencia del grafo de tipos, se define una enumeración que mapea cada tipo a un entero de 0 a 17:

```

1 typedef enum {
2     NORMAL=0, FIRE, WATER, ELECTRIC, GRASS, ICE, FIGHTING, POISON,
3     GROUND, FLYING, PSYCHIC, BUG, ROCK, GHOST, DRAGON,

```

```
4     DARK, STEEL, FAIRY    // 18 tipos, valores 0 a 17
5 } PokemonType;
6
7 #define NUM_TIPOS 18
```

Listing 3: Enumeración PokemonType en pokehash.h

3.2. Instancias y Selección de TDAs

El sistema declara cuatro instancias de TDAs que operan de forma coordinada:

TDA Mapa — Instancia 1: *Pokédex por Nombre* (pokedex) Se utiliza un TDA Mapa instanciado como **Pokédex por Nombre**, que indexa el catálogo completo usando como clave el campo **nombre** del Pokémon (tipo `char*`) y como valor un puntero a la entidad `Pokemon*`. **Justificación:** La búsqueda por nombre es la operación de consulta más frecuente. Un Mapa asocia la clave al valor en tiempo constante $O(1)$ promedio, eliminando la necesidad de recorrer secuencialmente los más de 1.100 registros.

TDA Mapa — Instancia 2: *Pokédex por ID* (pokedex.by_id) Se utiliza un segundo TDA Mapa instanciado como **Pokédex por ID**, que indexa el mismo catálogo usando como clave el ID numérico del Pokémon representado como cadena de texto (ej: "25" para Pikachu) y como valor el mismo puntero `Pokemon*` de la instancia anterior. **Justificación:** Sin este índice secundario, una búsqueda por número de Pokédex requeriría iterar todos los pares del mapa primario en $O(N)$. La indexación dual garantiza $O(1)$ promedio para búsquedas numéricas al costo de $O(N)$ de espacio adicional para las claves de texto del segundo mapa.

TDA Pila — Instancia: *Equipo Activo* (Equipo) Se utiliza un TDA Pila instanciado como **Equipo Activo**, que gestiona los Pokémon seleccionados en orden de inserción LIFO (Last-In, First-Out). Las operaciones de *push* (agregar) y *pop* (deshacer) son en $O(1)$. **Justificación:** La semántica LIFO es exactamente la que modela la funcionalidad de Deshacer: el último Pokémon en ser agregado es el primero en ser removido ante un error del usuario.

TDA Grafo Dirigido Ponderado — Instancia: *Red de Efectividades* (matrizDebilidades) Se utiliza un TDA Grafo Dirigido Ponderado instanciado como **Red de Efectividades de Tipos**. Cada nodo representa uno de los 18 tipos elementales y cada arista dirigida tiene como peso el multiplicador de daño ($0.0 =$ inmune, $0.5 =$ resistente, 1.0

= neutral, 2.0 = débil). **Justificación:** Las relaciones de efectividad son asimétricas y ponderadas (Fuego es efectivo contra Planta, pero Planta no lo es contra Fuego), lo que es la definición exacta de un Grafo Dirigido Ponderado. La consulta de efectividad entre cualquier par de tipos es directa e inmediata.

3.3. Estructuras de Datos Físicas

3.3.1. Mapas: Tabla Hash con Encadenamiento Separado

Ambos mapas (pokedex y pokedex.by_id) se implementan físicamente como una **Tabla Hash con Encadenamiento Separado** (*Separate Chaining*). El núcleo de la estructura, definido en `tdas/map.c`, es:

```

1 #define BUCKETS_LIMIT 200
2
3 struct Map {
4     List *buckets[BUCKETS_LIMIT]; // Arreglo estatico de 200 listas
    // enlazadas simples
5     int size;                      // Numero total de pares
    // almacenados
6     int (*is_equal)(void*, void*); // Funcion de comparacion de
    // claves inyectada
7     int current_bucket;           // Bucket activo durante la
    // iteracion
8 };

```

Listing 4: Estructura interna de la Tabla Hash en `tdas/map.c`

La función de dispersión implementada es **DJB2** (Dan Bernstein Hash 2). Para lograr búsquedas insensibles a mayúsculas, cada carácter se normaliza a minúsculas antes de incorporarse al hash:

```

1 static unsigned long hash_string(void *key) {
2     char *str = (char *)key;
3     unsigned long hash = 5381;
4     int c;
5     while ((c = (unsigned char)*str++))
6         hash = ((hash << 5) + hash) + tolower(c); /* hash * 33 +
    // tolower(c) */
7     return hash;
8 }

```

Listing 5: Función de hash DJB2 con normalización a minúsculas en `tdas/map.c`

El índice del bucket se calcula como $\text{bucket} = \text{hash_string}(\text{clave}) \bmod 200$. Cada bucket es una **Lista Enlazada Simple** (`tdas/list.c`) que almacena pares `MapPair{key, value}` para resolver colisiones. La comparación de claves dentro del bucket usa `strcasecmp` (inyectada en la creación del mapa como `string_is_equal`), garantizando que "Pikachu", "pikachu" y "PIKACHU" se traten como la misma clave.

Justificación física: Con 200 buckets y ~ 1.100 Pokémon, el factor de carga $\alpha \approx 5.5$. En el caso promedio, la búsqueda requiere $O(1)$ saltos de hashing más $O(\alpha)$ comparaciones de cadena, que para α constante es $O(1)$. La Lista Enlazada Simple en cada bucket añade un overhead mínimo de memoria ($O(N)$ nodos totales) sin sacrificar la complejidad de acceso.

3.3.2. Pila del Equipo: Arreglo Estático de Punteros

La Pila del equipo se implementa físicamente como un **arreglo estático de 6 punteros** con un entero índice `tope`. Las operaciones físicas son:

- **push** (agregar): `integrantes[tope] = p; tope++;` $\Rightarrow O(1)$ exacto.
- **pop** (deshacer): `tope--;` $\Rightarrow O(1)$ exacto. El puntero anterior en `integrantes[tope]` no se destruye; simplemente queda fuera del alcance lógico de la pila.
- **top** (ver cima): `integrantes[tope-1];` $\Rightarrow O(1)$ exacto.

Justificación física: El tamaño máximo del equipo es **exactamente 6**, por regla inamovible del juego. Usar memoria dinámica (`malloc/realloc`) añadiría overhead de gestión de heap sin ningún beneficio funcional. El arreglo estático de 6 posiciones es la estructura más eficiente posible: cero overhead de punteros extras, acceso en un único ciclo de reloj.

3.3.3. Grafo de Tipos: Matriz de Adyacencia Estática

El Grafo de Efectividades se implementa físicamente como una **Matriz de Adyacencia Estática** de tipo `float[18][18]`, declarada en `pokehash.c`:

```
1 float matrizDebilidades[NUM_TIPOS][NUM_TIPOS]; /* Matriz 18x18 = 324
    celdas float */
```

Listing 6: Declaración de la Matriz de Adyacencia en `pokehash.c`

Cada celda `matrizDebilidades[i][j]` almacena el multiplicador de daño cuando un ataque de tipo i impacta a un Pokémon de tipo j . La matriz es cargada en tiempo

de inicio desde `Tabla.csv` mediante `cargar_matriz_debilidades()`, que la puebla fila por fila usando `strtok` y `atof`.

Justificación física: La consulta de efectividad entre dos tipos es un acceso directo por índice doble (`matrizDebilidades[i][j]`) en $O(1)$ absoluto, sin recorridos. Una Lista de Adyacencia requeriría recorrer la lista de aristas del nodo i para encontrar el nodo j , con costo $O(T)$ donde $T = 18$ en un grafo completo. La Matriz de Adyacencia es la estructura física óptima para grafos densos con consultas frecuentes de aristas individuales.

4. Diseño de la Solución Algorítmica

En esta sección se describe la lógica de cada funcionalidad usando exclusivamente las operaciones formales de los TDAs, sin mencionar punteros ni índices internos.

4.1. Lógica Operacional por Funcionalidad

4.1.1. Cargar Catálogo (`cargar_data`)

1. Se inicializan las instancias **Pokédex por Nombre** y **Pokédex por ID** con la operación `crear(comparador)`.
2. Se abre el archivo `Pokemon.csv` y se descarta la primera línea (cabecera).
3. Por cada registro del archivo:
 - a) Se crea una nueva entidad `Pokemon` con sus campos parseados.
 - b) Se genera el nombre único aplicando las reglas de composición de forma regional.
 - c) Se aplica `insertar(nombre, Pokemon)` sobre la **Pokédex por Nombre**.
 - d) Se convierte el ID entero a cadena de texto y se aplica `insertar(id_texto, Pokemon)` sobre la **Pokédex por ID**.
4. Se repite hasta leer todos los registros. La operación `insertar` ignora automáticamente los duplicados, por lo que Pokémon con formas regionales que comparten ID solo quedan indexados por el nombre de su primera aparición en el segundo mapa.

4.1.2. Cargar Matriz de Efectividades (`cargar_matriz_debilidades`)

1. Se abre `Tabla.csv` y se descarta la cabecera (fila de nombres de tipos).
2. Por cada una de las 18 filas, se omite la primera columna (nombre del tipo atacante) y se leen los 18 valores numéricos `float` que representan la efectividad del tipo i contra cada uno de los 18 tipos defensivos j .
3. Cada valor se almacena en la celda `matrizDebilidades[i][j]` de la **Red de Efectividades**.

4.1.3. Buscar por Nombre (`buscar_por_nombre`)

1. Se lee la cadena ingresada por el usuario. Se eliminan espacios al inicio y final (*trim*) y el salto de línea residual.
2. Se aplica la operación `buscar(nombre)` sobre la **Pokédex por Nombre**.
3. Si la operación retorna un resultado válido, se despliega la ficha técnica completa del Pokémon. En caso contrario, se notifica al usuario que no fue encontrado.

4.1.4. Buscar por Número de Pokédex (`buscar_por_id`)

1. Se lee el entero ingresado por el usuario con validación de formato.
2. Se convierte el entero a su representación textual (ej: $25 \rightarrow "25"$).
3. Se aplica la operación `buscar(id_texto)` sobre la **Pokédex por ID**.
4. Se muestra la ficha técnica o el mensaje de error según el resultado.

4.1.5. Buscar con Filtro por Generación y Tipo (`buscar_por_filtro_gen_tipo`)

1. Se leen la generación (entero) y el tipo elemental (cadena) ingresados por el usuario.
2. Se inicia la iteración sobre la **Pokédex por Nombre** con la operación `primero()`.
3. Por cada `Pokemon` obtenido con `siguiente()`:
 - a) Se verifica si `Pokemon.gen` coincide con la generación buscada.
 - b) Se verifica si `Pokemon.tipo1` o `Pokemon.tipo2` coincide con el tipo buscado (insensible a mayúsculas).

- c) Si ambas condiciones se cumplen, se imprime la fila del Pokémon.
- 4. Al terminar la iteración, se imprime el total de coincidencias.

4.1.6. Analizar Debilidades del Equipo (analizar_debilidades)

1. Se verifica que el **Equipo Activo** tenga al menos un integrante (`tope > 0`); en caso contrario se notifica y se retorna.
2. Se declara un arreglo estático `conteo_debilidades_equipo[NUM_TIPOS]`, inicializado en cero, para acumular cuántos integrantes son débiles a cada tipo.
3. Para cada uno de los k Pokémon del equipo ($i = 0 \dots \text{tope} - 1$):
 - a) Se obtiene el índice de `tipo1` llamando a `obtener_indice_tipo()`, que recorre linealmente el arreglo `NOMBRES_TIPOS[18]` con `strcasecmp`: $O(T) = O(1)$.
 - b) Para cada tipo atacante j (0 a 17), se calcula el multiplicador efectivo de daño consultando la **Red de Efectividades**: `multiplicador = matrizDebilidades[j][tipo1] × matrizDebilidades[j][tipo2]`, acceso en $O(1)$ absoluto.
 - c) Si el multiplicador $> 1,0$, se imprime la debilidad (indicando si es $\times 2$ o $\times 4$ crítico) y se incrementa `conteo_debilidades_equipo[j]`.
4. Tras el bucle de integrantes, se recorre el arreglo de conteo (18 posiciones) para emitir alertas globales por tipos compartidos: $O(T) = O(1)$.

4.2. Algoritmo Core: Ranking Top 10 (mejores10PorStat)

Este es el algoritmo de mayor complejidad lógica del sistema. Combina el recorrido del mapa con el mantenimiento de un **ranking de top- K en tiempo lineal**:

1. El usuario especifica el modo (1 stat o suma de 2 stats) y un filtro opcional (toda la Pokédex, por generación, o por tipo).
2. Se inicializa un **Arreglo Estático Ordenado Descendentemente de $K = 10$ posiciones**, con puntajes iniciales de -1 .
3. Se inicia la iteración sobre la **Pokédex por Nombre**:
 - a) Si el Pokémon no cumple el filtro activo, se avanza sin procesarlo.
 - b) Se calcula el puntaje del Pokémon (stat seleccionada, o suma de dos stats).

- c) **Si el puntaje es mayor que el décimo lugar del arreglo:** se busca la posición correcta de inserción recorriendo el arreglo de manera descendente, se desplazan los elementos hacia abajo y se inserta el nuevo Pokémon. Este proceso toma $O(K) = O(10) = O(1)$ por ser K constante.
4. Tras recorrer los N Pokémon del catálogo, el arreglo contiene el Top-10 ordenado, que se imprime.

Por qué no se usa el TDA Heap: El TDA Heap disponible (`tdas/heap.h`) implementa un *max-heap*. Usarlo para resolver un problema Top- K requeriría insertar los N Pokémon y luego extraer los 10 mejores, con costo $O(N \log N)$. El algoritmo de ventana estática con $K = 10$ tiene costo $O(N \cdot K) = O(N)$ al ser K constante, lo que lo hace **asintóticamente superior** para este problema.

5. Análisis de Complejidad

5.1. Tabla de Complejidad Temporal y Espacial

En la siguiente tabla se presenta el análisis de peor caso (*Worst-Case*) de todas las funciones del sistema. N es el número total de Pokémon (~ 1.100), k el número de integrantes del equipo ($0 \leq k \leq 6$), α el factor de carga del hash ($\approx 5,5$) y $T = 18$ el número de tipos elementales.

Tabla 1: Complejidad de las funciones de PokeHash

Función	Caso Prome- dio	Peor Caso	Esp. Adicio- nal
<code>cargar_data()</code>	$O(N)$	$O(N)$	$O(N)$
<code>cargar_matriz_debilidades()</code>	$O(T^2) = O(1)$	$O(1)$	$O(1)$
<code>buscar_por_nombre()</code>	$O(1)$	$O(N)^*$	$O(1)$
<code>buscar_por_id()</code>	$O(1)$	$O(N)^*$	$O(1)$
<code>buscar_por_filtro_gen_tipo()</code>	$O(N)$	$O(N)$	$O(1)$
<code>mejores10PorStat()</code>	$O(N)$	$O(N)$	$O(1)$
<code>agregar_pokemon_equipo()</code>	$O(1)$	$O(N)^*$	$O(1)$
<code>analizar_debilidades()</code>	$O(k \cdot T^2) = O(1)$	$O(1)$	$O(1)$
<code>deshacer()</code>	$O(1)$	$O(1)$	$O(1)$
<code>ver_equipo_actual()</code>	$O(k) = O(1)$	$O(1)$	$O(1)$
<code>liberar_pokedex()</code>	$O(N)$	$O(N)$	$O(1)$

* El peor caso teórico $O(N)$ ocurre solo si la función hash genera colisión total (todos los N elementos en el mismo bucket). En la práctica con DJB2 y 200 buckets esto es extremadamente improbable.

5.2. Justificación de Bajo Nivel

5.2.1. `buscar_por_nombre()` y `buscar_por_id()`: $O(1)$ promedio

El proceso interno de `map_search` ejecuta exactamente los siguientes pasos:

1. **Hashing:** Se ejecuta DJB2 sobre la clave, iterando carácter a carácter. La longitud máxima de una clave es 100 caracteres (nombre) o 4 caracteres (ID), ambos constantes $\Rightarrow O(1)$.
2. **Indexación:** Se calcula `bucket = hash % 200`, una operación de módulo $\Rightarrow O(1)$.
3. **Acceso al bucket:** Acceso directo a `buckets[bucket]`, indexación de arreglo $\Rightarrow O(1)$.

4. **Recorrido de la cadena:** Se recorre la Lista Enlazada del bucket comparando claves con `strcasemp`. Con factor de carga $\alpha \approx 5,5$, se realizan en promedio $O(\alpha) \approx O(6)$ comparaciones $\Rightarrow O(1)$ constante. En el peor caso teórico (colisión total), se recorren los N elementos de la cadena $\Rightarrow O(N)$.

5.2.2. `buscar_por_filtro_gen_tipo()`: $O(N)$ inherente e irreducible

Esta función usa `map_first()` y `map_next()` para iterar todos los pares del mapa. Internamente, `map_next()` avanza en la lista del bucket actual ($O(1)$) o busca el siguiente bucket no vacío en el arreglo de 200 buckets ($O(200/N) \approx O(1)$). El costo dominante es recorrer los N pares: $O(N)$.

Esta complejidad es **inherente al diseño**: la Tabla Hash está indexada solo por nombre. Para filtrar por tipo y generación simultáneamente no existe un atajo posible sin un índice adicional (por ejemplo, un tercer mapa indexado por tipo). Con las estructuras actuales, $O(N)$ es la cota inferior para esta operación.

5.2.3. `mejores10PorStat()`: $O(N)$ con ventana de inserción constante

El algoritmo mantiene un arreglo ordenado de $K = 10$ posiciones. Por cada uno de los N Pokémon del catálogo:

- Se compara su puntaje con el décimo lugar: $O(1)$.
- Si supera el mínimo (caso frecuente para los primeros elementos, cada vez más raro a medida que el top-10 se consolida): búsqueda lineal en el arreglo de $K = 10$ posiciones más desplazamiento: $O(K) = O(10) = O(1)$.

Total: N iteraciones de $O(1) = O(N)$. La complejidad espacial es $O(1)$ ya que el arreglo de 10 posiciones es estático y no escala con N .

5.2.4. `deshacer()`: $O(1)$ estricto

La operación de deshacer ejecuta exactamente una instrucción de decremento aritmético sobre el entero `tope`. No accede a memoria dinámica, no recorre ninguna estructura, no llama a `free()`. La complejidad es $O(1)$ exacto en todos los casos posibles.

5.2.5. `analizar_debilidades()`: $O(1)$ con $k \leq 6$ y $T = 18$ constantes

La función ejecuta tres bucles anidados, pero todos acotados por constantes del dominio del juego:

- **Bucle externo** sobre integrantes: $k \leq 6$ iteraciones.
- **Bucle interno** sobre tipos atacantes: $T = 18$ iteraciones fijas.
- **Por cada par** (i, j) : dos llamadas a `obtener_indice_tipo` (recorrido lineal de 18 elementos $= O(T) = O(1)$) y dos accesos directos a `matrizDebilidades[j][tipo]` ($O(1)$ absoluto).

Costo total: $O(k \cdot T \cdot T) = O(6 \cdot 18 \cdot 18) = O(1944) = \mathbf{O(1)}$ exacto. Ninguna variable escala con el tamaño del catálogo N . El arreglo auxiliar `conteo_debilidades_equipo[18]` tiene tamaño fijo $\Rightarrow O(1)$ espacial. **Observación:** `obtener_indice_tipo` se llama dentro del bucle interno, lo que genera hasta $2 \times 18 = 36$ llamadas redundantes por Pokémon. Una optimización directa es precomputar ambos índices antes del bucle j , reduciendo las llamadas de $O(k \cdot T)$ a $O(k)$ sin alterar la complejidad asintótica (ambas son $O(1)$), pero mejorando el rendimiento constante.

5.2.6. `liberar_pokedex()`: $O(N)$ con doble responsabilidad de propiedad

La liberación requiere $O(N)$ por la doble responsabilidad de propiedad de memoria:

1. **Primera iteración** sobre la **Pokédex por Nombre**: se libera cada objeto `Pokemon*` (N operaciones `free`) y se destruye el mapa (libera 200 buckets y el struct `Map`): $O(N) + O(1)$.
2. **Segunda iteración** sobre la **Pokédex por ID**: se libera cada clave dinámica `id_key` (N operaciones `free`). Los valores `Pokemon*` **no** se liberan aquí (ya fueron liberados en el paso anterior): $O(N) + O(1)$.

Total: $O(N)$. Este orden es **obligatorio**: liberar los `Pokemon*` desde el segundo mapa también causaría un error de doble liberación (*double-free*).

6. Aspecto Desafiante

El desarrollo de PokeHash presentó tres desafíos técnicos de alto nivel en C:

6.1. Gestión Manual de Memoria Dinámica con Doble Modelo de Propiedad

En C no existe recolector de basura. Cada una de las ~ 1.100 entidades `Pokemon` se asigna individualmente con `malloc` durante la carga:

```

1 Pokemon *pmon = (Pokemon *)malloc(sizeof(Pokemon));
2 /* ...se rellenan los campos del struct... */
3 map_insert(pokedex, pmon->nombre, pmon);

```

Listing 7: Asignación dinámica por Pokémon en cargar_data()

Adicionalmente, para el segundo mapa se asigna una clave de texto dinámica por cada Pokémon:

```

1 char *id_key = (char *)malloc(20 * sizeof(char));
2 snprintf(id_key, 20, "%d", pmon->id);
3 map_insert(pokedex_by_id, id_key, pmon); /* La clave es dinamica; el
   valor no */

```

Listing 8: Asignación dinámica de clave ID para el segundo mapa

El desafío reside en que ambos mapas comparten los mismos `Pokemon*`, pero con **modelos de propiedad asimétricos**: el primer mapa es propietario de los objetos `Pokemon` (debe liberarlos con `free`), mientras que el segundo mapa es propietario únicamente de sus claves `id_key` (debe liberar esas cadenas, pero **no** los valores `Pokemon*`, que ya fueron liberados). Una confusión en este orden produce doble liberación (*double-free*) o fugas de memoria. La función `liberar_pokedex()` implementa correctamente esta secuencia:

```

1 /* 1. Libera objetos Pokemon (propiedad del mapa primario) */
2 MapPair *pair = map_first(pokedex);
3 while (pair != NULL) {
4     free((Pokemon *)pair->value); /* Solo el valor, NO la clave (
   nombre del struct) */
5     pair = map_next(pokedex);
6 }
7 map_destroy(pokedex); /* Libera nodos de lista, buckets y struct
   Map */
8
9 /* 2. Libera las claves dinamicas de ID (propiedad del mapa
   secundario) */
10 pair = map_first(pokedex_by_id);
11 while (pair != NULL) {
12     free((char *)pair->key); /* Solo la clave id_key, NO el valor
   Pokemon* */
13     pair = map_next(pokedex_by_id);
14 }
15 map_destroy(pokedex_by_id);

```

Listing 9: Liberación correcta y ordenada en liberar_pokedex()

6.2. Diseño de la Función Hash con Insensibilidad a Mayúsculas

Lograr que `charizard`, `Charizard` y `CHARIZARD` calculen el mismo bucket requirió modificar la función DJB2 para incorporar `tolower(c)` en cada iteración antes de acumular el hash. Complementariamente, la función de comparación inyectada en el mapa usa `strcasecmp` en lugar de `strcmp`:

```
1 int string_is_equal(void *key1, void *key2) {  
2     if (key1 == NULL || key2 == NULL) return 0;  
3     return strcmp((char *)key1, (char *)key2) == 0;  
4 }
```

Listing 10: Comparación insensible a mayúsculas en `pokehash.c`

Esta combinación — hashing con normalización + comparación insensible — garantiza que el mismo Pokémon sea encontrado independientemente de cómo el usuario capitalice su nombre.

6.3. Coordinación Precisa de la Indexación Dual

Mantener dos Tablas Hash que referencian los mismos objetos en memoria exige una coordinación sin errores:

- Ambas inserciones deben ocurrir en la misma iteración del ciclo de carga, con el mismo `Pokemon*`.
- La verificación interna de duplicados de `map_insert` garantiza que si la inserción en el primer mapa falla (nombre ya existe), tampoco se inserta en el segundo. Esto es crítico para Pokémon con formas regionales que comparten nombre base.
- Pokémon de la misma especie pero distintas formas (ej: `Charizard` e `"Mega Charizard X"`) comparten el mismo ID numérico. Esto implica que `pokedex.by_id` solo indexa la *primera* forma encontrada para ese ID, ya que la segunda inserción es rechazada por duplicado. Este comportamiento es determinista y esperado.

7. Planificación y Contribución del Equipo

7.1. Distribución de Roles y Responsabilidades

Coordinador de Proyecto — Cristóbal Sazo Responsable de la arquitectura global del sistema. Definió el diseño de la indexación dual con dos Tablas Hash, supervisó

la lógica de gestión de memoria dinámica (modelo de doble propiedad) y resolvió los cuellos de botella de integración entre módulos.

Comunicador — Sebastián Riveros Responsable de la documentación formal del proyecto. Redactó el Informe de Avance y el Informe Final, documentó los TDAs y sus justificaciones de complejidad, y actuó como enlace oficial con la cátedra para resolver dudas de implementación.

Gestor de Calidad — Simón Guzmán Responsable de la robustez técnica y la metodología ágil. Supervisó el control de versiones Git, realizó revisiones cruzadas de código para detectar accesos a punteros nulos y fugas de memoria, y validó que el manejo de errores en lectura de CSV y entradas del usuario fuera robusto.

Responsable de Integración — Bruno Orellana Supervisor final de la arquitectura. Coordinó la unión de los módulos de carga de archivos, estructuras de datos y menús. Lideró las pruebas de estrés (datos erróneos, nombres inexistentes, equipo lleno) y auditó la consistencia de nomenclatura en todo el código fuente.

7.2. Lista de Tareas Realizadas por Integrante

Tabla 2: Distribución de tareas del proyecto PokeHash

Integrante	Tareas Realizadas
Cristóbal Sazo	Diseño de la arquitectura dual de mapas hash. Implementación de <code>cargar_data()</code> con doble inserción. Implementación de <code>liberar_pokedex()</code> con doble iteración y modelo de propiedad asimétrico. Resolución de bugs de gestión de memoria.
Sebastián Riveros	Redacción del Informe de Avance e Informe Final. Documentación técnica de TDAs, complejidades y estructuras físicas. Implementación de la función de comparación insensible a mayúsculas (<code>string_is_equal</code> con <code>strcasecmp</code>). Generación del <code>README.md</code> del repositorio.
Simón Guzmán	Gestión del repositorio Git (ramas, merges y resolución de conflictos). Implementación de <code>menu_explorar_pokedex()</code> y sus tres subfunciones de búsqueda. Validación de entradas del usuario (limpieza de buffer, trim de strings, manejo de EOF). Pruebas de integración y manejo de errores.
Bruno Orellana	Implementación de <code>mejores10PorStat()</code> con algoritmo de ventana Top-10. Implementación de <code>cargar_matriz_debilidades()</code> desde <code>Tabla.csv</code> . Implementación del módulo de gestión de equipo (<code>agregar_pokemon_equipo</code> , <code>ver_equipo_actual</code> , <code>deshacer</code>). Pruebas de estrés con datos inválidos y casos borde.

7.3. Carta Gantt

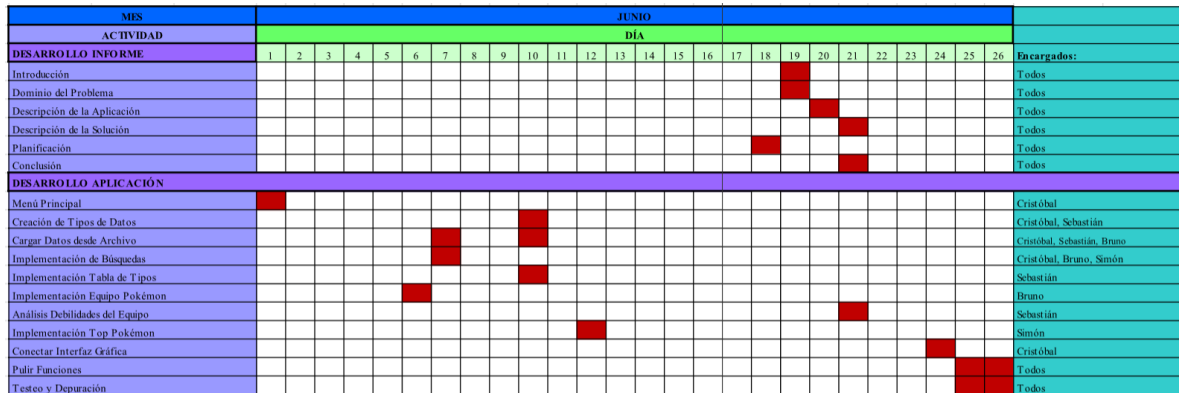


Figura 1: Carta Gantt — Planificación del desarrollo de PokeHash (Junio)

8. Conclusión

El desarrollo de PokeHash permitió aplicar de forma integrada los conceptos fundamentales de las Estructuras de Datos en C, resultando en una aplicación funcional y de alto rendimiento computacional para la gestión táctica de equipos Pokémon.

La decisión arquitectónica más relevante fue la **indexación dual**: dos Tablas Hash operando en paralelo, una por nombre y otra por ID. Esta decisión elevó la búsqueda por número de Pokédex de $O(N)$ (recorrido lineal del mapa primario) a $O(1)$ promedio, a costa de un espacio adicional de $O(N)$ — completamente justificado dado que las claves del segundo mapa son cadenas de máximo 4 caracteres (IDs de hasta 4 dígitos).

La modelización del sistema de tipos como un **Grafo Dirigido Ponderado con Matriz de Adyacencia estática** demostró ser la elección ideal para un grafo denso y de tamaño fijo ($18 \times 18 = 324$ celdas). Las consultas de efectividad son en $O(1)$ absoluto, sin recorridos. En contraste, una Lista de Adyacencia para el mismo grafo denso habría requerido hasta $O(18)$ comparaciones por consulta.

La **Pila estática** de 6 posiciones para el equipo fue la solución perfecta para un tamaño fijo y operaciones LIFO: agregar y deshacer son $O(1)$ exacto sin overhead de memoria dinámica.

Como debilidad a reconocer, la función de filtrado por generación y tipo requiere un recorrido completo de $O(N)$ porque la Tabla Hash primaria está indexada únicamente por nombre. Una mejora futura consistiría en agregar un Mapa Múltiple indexado por tipo elemental (aprovechando el TDA `multimap.h` disponible en el proyecto), que permitiría filtrar en $O(\text{resultados})$ sin revisar el catálogo completo.

En síntesis, PokeHash demuestra el dominio práctico de cuatro estructuras fundamentales — Tabla Hash con Encadenamiento, Lista Enlazada Simple, Pila con Arreglo Estático y Grafo con Matriz de Adyacencia — y su integración coordinada en un sistema de búsqueda de alta eficiencia, con gestión rigurosa de memoria dinámica y cero fugas (*zero memory leaks*) al finalizar la ejecución.